ЕДИНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ(SRC) ДЛЯ ПРОЕКТА “Т-ПУТЕШЕСТВИЯ”

1. ОБЩИЕ ТРЕБОВАНИЯ И ГРАНИЦЫ MVP.

- **Язык и валюта**: интерфейс на русском языке. Все финансовые расчеты в системы производятся в рубли. Мультивалютность в MVP отсутствует.
- **Стек технологий**: React, .NET Web Api, PostgreSQL, Python FastAPI.
- **Архитектурный подход**: Разработка базы данных ведется по методологии Code-First через Entity Framework.
- Наличие переключателей Светлой/Темной темы на всех экранах.
- Платежи: приложение не интегрируется с банковскими API. Ссылка Т-банка используется для перехода к переводу; факт оплаты отмечается вручную пользователем

2. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ И СПЕЦИФИКАЦИЯ ЭКРАНОВ

2.1 Контур авторизации и профиля(Экран 1)

- Регистрация и вход осуществляются исключительно по номеру телефона с одноразовым SMS-кодом (OTP). Номер телефона обязателен для зарегистрированного пользователя и уникален. Email — необязательное поле личного кабинета; email/пароль и отдельные пароли в MVP не используются.
- Сброс пароля и функция «Забыли пароль?» в MVP отсутствуют, поскольку парольная авторизация исключена. Повторный вход осуществляется по SMS.
- **Реквизиты оплаты:** В профиле пользователя присутствует текстовое поле (tbank_payment_url) для хранения персональной ссылки-запроса на перевод из приложения Т-Банка физлица.

2.2 Главный экран / Дашборд (Экран 2)

- **Виджет быстрого баланса:** Отображает общее суммарное сальдо пользователя по всем его поездкам: «Мне должны всего: XXX ₽» (подсвечивается зеленым) и «Я должен всего: YYY ₽» (подсвечивается красным/серым).
- **Список поездок:** Карточки поездок выводятся списком и фильтруются бэкендом на три визуальные категории: Активные (текущая дата внутри диапазона), Предстоящие (дата начала в будущем), Прошедшие (архивные поездки, дата завершения в прошлом).
- **Строка поиска:** В MVP поиск по поездкам отсутствует с целью оптимизации ресурсов разработки.

2.3 Создание поездки и приглашения (Экран 3 – Модальное окно)

- **Поля ввода:** Название поездки, Даты (интерактивный календарь-диапазон).
- **Приглашение друзей:** Ввод Email или Номера телефона друга и кнопка «Пригласить». В таблице участников запись создается со статусом Pending (Ожидает подтверждения). Приглашение доступно только для пользователей, уже зарегистрированных в системе. Если пользователь с указанным email или номером телефона не найден, система сообщает о невозможности отправить приглашение.
- **Логика получателя:** Приглашенный пользователь видит всплывающее уведомление/шторку со спецификацией организатора и кнопками «Принять» (статус меняется на Accepted) или «Отказаться» (статус меняется на Declined, доступ закрывается).
- **Виртуальные пользователи:** Если участник не имеет приложения или не хочет регистрироваться, организатор может добавить его как виртуального пользователя (в БД создается запись без привязки к user_id, но с заполненным полем имени).

2.4 **Экран «Карточка поездки» (Основное рабочее пространство)**

- **Вкладка «Лента расходов»:** Список всех чеков, отсортированных от новых к старым. Карточка расхода визуализирует: кто платил, сколько (в рублях), категорию (иконка), дату операции и миниатюру прикрепленного чека.
- **Форма «Добавить расход» (Модальное окно поверх ленты):** «Добавить расход»: ручной ввод суммы, названия, категории (Еда, Транспорт, Жильё, Развлечения, Другое), плательщика из участников поездки, долей «Поровну» или «Своя сумма», загрузка фото чека и OCR-предзаполнение. Сумма долей должна совпадать с общей суммой. Самостоятельное указание долей участниками не входит в обязательный MVP (см. раздел 3.7).
- **Вкладка «Итоги (Сальдо)»:** Блок «Кто кому сколько должен» на базе алгоритма минимизации транзакций (схлопывание цепочек долгов). Напротив каждой строки долга выведена кнопка быстрого расчета «Оплатить долг», перенаправляющая на tbank_payment_url кредитора. Долг в MVP гасится только полностью в один клик. Для зарегистрированных участников подтверждение погашения долга доступно только самому должнику через кнопку «Я оплатил». Если должником является виртуальный участник, подтверждение погашения выполняет организатор поездки через кнопку «Отметить как оплаченный». После подтверждения информация о погашении сохраняется в таблице Logs с типом действия DebtSettled. Подтверждённые переводы учитываются при последующем расчёте сальдо. Частичное погашение долгов в MVP не поддерживается.
- **Вкладка «Детали поездки»:** таймлайн событий и мест. Базовый MVP: поиск предложений авиабилетов по маршруту и дате через Travelpayouts, переход по внешней ссылке для покупки и ручное добавление выбранного рейса в таймлайн (без заявления о подтверждённой покупке). Импорт

сведений по ссылкам на отели и достопримечательности остаётся отдельным сценарием Python-парсера. Автоматическое распознавание покупки и добавление подтверждённого бронирования — только расширенный этап после проверки атрибуции и содержимого статистики. ИИ-чат отсутствует.

- **Вкладка «Аудит (История)»:** Простая текстовая хронология логов всех изменений действий участников (например: «Иван изменил сумму расхода 'Ужин' с 500 на 700 руб. 20 мин. назад»).

3. **СИСТЕМНЫЕ ТРЕБОВАНИЯ И ИНТЕГРАЦИОННЫЕ КОНТРАКТЫ**

3.1 Схема взаимодействия Backend-developers (.NET Core – Python FastAPI)

Интеграция умного распознавания чеков(OCR) строится по следующей схеме:

1. React-клиент отправляет файл фотографии чека на основной .NET API.
2. .NET API пересылает данный файл через HttpClient в Python микросервис.
3. Python-микросервис обрабатывает изображение с помощью специализированных библиотек оптического распознавания символов (OCR-инструментов), передает полученный сырой текст в API языковой модели с системным промптом структурирования и возвращает JSON обратно в .NET.
4. .NET бэкенд верифицирует данные и возвращает этот JSON на фронтенд для предзаполнения формы добавления расхода.

3.2 **Спецификация API контрактов(Протокол Frontend – Backend)**

Общие соглашения: JSON в camelCase; ID участника поездки — memberId, ID зарегистрированного аккаунта — userId. Во всех денежных полях используются рубли и два знака после запятой. Маршруты с {tripId} требуют проверки доступа к поездке. Примеры ниже фиксируют интерфейс MVP

Контракт 1: Распознование чека через ИИ(OCR)

Эндпоинт: POST /api/expenses/recognize

Тип данных: multipart/form-data (Передача файла изображения ‘file’)

Ответ от сервера (200)

```
{
  "suggestedAmount": 3450.00,
  "suggestedTitle": "Ресторан Сыроварня",
  "suggestedCategory": "Food"
}
```

Контракт 2: Сохранение расхода (Умное деление трат)

Эндпоинт: POST /api/trips/{tripId}/expenses

Content-Type: application/json. Виртуальные участники заранее создаются через members/virtual и передаются обычным memberId

Тело запроса (Application/JSON):

```
{
  "paidByMemberId": 1,
```

```
  "title": "Ужин в ресторане",
  "category": "Food",
  "totalAmount": 3000.00,
  "splits": [
   { "memberId": 1, "shareAmount": 1000.00},
   { "memberId": 2, "shareAmount": 1500.00},
   { "memberId": 3, "shareAmount": 500.00}
  ]
}
```

Проверки: paidByMemberId и все memberId принадлежат tripId; amount > 0; shareAmount ≥ 0; сумма splits = totalAmount; один memberId не повторяется в одном расходе. Ответ: HTTP 201 с expenseId.

```
{
 "expenseId": 74
}
```

Контракт 3: Получение оптимизированных долгов (Алгоритм Сальдо)

Эндпоинт: GET /api/trips/{tripId}/balances
Ответ от сервера (200):

```
{
  "tripId": 12,
  "optimizedTransfers": [
   {
    "fromMemberId": 2,
    "fromMemberName": "Петр",
    "toMemberId": 1,
    "toMemberName": "Иван",
    "amount": 1500.00,
    "targetTbankUrl": "https://www.tinkoff.ru/rm/ivan_travel/wQ1zX4"
   }
  ]
}
```

Алгоритм вычисляет сальдо по Expenses/Expense_splits, учитывает подтвержденные DebtSettled из Logs, затем оптимизирует переводы.

Контракт 4: Импорт развлечений и деталей поездки по ссылке

Эндпоинт: POST /api/trips/{tripId}/activities/import
Тело запроса (Application/JSON):

```
{
  "sourceUrl": "https://kudago.com/spb/place/kazanskij-sobor/"
}
```

Ответ от сервера (200):

```
{
  "activityId": 45,
  "title": "Казанский собор",
  "type": "Sight",
  "description": "Один из крупнейших храмов Санкт-Петербурга, выполненный в стиле ампир.",
  "location": "Казанская площадь, 2, Санкт-Петербург"
}
```

Контракт 5: Добавление виртуального участника

Эндпоинт: POST /api/trips/{tripId}/members/virtual
Тело запроса (Application/JSON):

```
{
  "virtualName": "Коля без приложения"
}
```

Ответ HTTP (201):

```
{
"memberId": 3,
  "virtualName": "Коля без приложения",
  "role": "Member",
  "status": "Accepted"
}
```

Контракт 6: Список участников

Эндпоинт: GET /api/trips/{tripId}/members

Ответ HTTP (200):

```
{
  "members": [
    {
      "memberId": 1,
      "userId": 9,
      "displayName": "Иван",
      "isVirtual": false,
      "role": "Owner",
      "status": "Accepted"
    },
    {
      "memberId": 3,
      "userId": null,
      "displayName": "Коля без приложения",
      "isVirtual": true,
      "role": "Member",
      "status": "Accepted"
    }
  ]
}
```

Контракт 7: Подтверждение погашения долга

Эндпоинт: POST /api/trips/{tripId}/balances/settle

Запрос на подтверждение погашения долга отправляет зарегистрированный должник либо организатор поездки, если должник является виртуальным участником. Запрос требует авторизацию и заголовок Idempotency-Key с уникальным ключом операции; ключ должен надежно сохраняться или иначе обеспечиваться от повторной обработки.

Тело запроса (Application/JSON):

```
{
  "fromMemberId": 2,
  "toMemberId": 1,
  "amount": 1500.0
}
```

Ответ HTTP (200):

```
{
  "status": "Confirmed",
  "tripId": 12,
  "fromMemberId": 2,
  "toMemberId": 1,
  "amount": 1500.0
}
```

Сервер сверяет сумму с текущим оптимизированным переводом и проверяет права инициатора: зарегистрированный должник подтверждает собственный платёж, а за виртуального должника подтверждение выполняет организатор поездки, и в одной транзакции создаёт неизменяемую запись Logs с action_type = DebtSettled. Повтор того же запроса не должен повторно списывать баланс; при изменившемся долге возвращается 409 Conflict.

Правила авторизации и подтверждения оплаты:

1. Если fromMemberId принадлежит зарегистрированному пользователю, подтвердить оплату может только этот пользователь.
2. Если fromMemberId принадлежит виртуальному участнику (user_id = NULL), подтвердить оплату может только организатор поездки (role = Owner).
3. Бэкенд проверяет права пользователя по JWT-токену и принадлежность участников указанной поездке.
4. Сумма подтверждения должна соответствовать полной сумме актуального оптимизированного перевода.
5. После успешного подтверждения создаётся запись DebtSettled в таблице Logs.
6. При отсутствии необходимых прав сервер возвращает HTTP 403 Forbidden.

3.3 Базовые API- маршруты и уточнения

Модуль	Метод и маршрут	Назначение/Уточнение
Auth	POST /api/auth/phone/send-code	Отправка одноразового SMS/пуш-кода.

Auth	POST /api/auth/phone/verify-code	Проверка кода, создание/вход в аккаунт и выдача JWT.
Trips	GET /api/trips	Список поездок с вычисляемым статусом Active/Future/Past.
Trips	POST /api/trips	Создание поездки (имя, даты, обложка); currency отсутствует.
Members	POST /api/trips/{tripId}/invite	Приглашение через email или телефон.
Members	PUT /api/trips/{tripId}/invite/respond	Ответ Accept/Decline на приглашение.
Members	POST /api/trips/{tripId}/members/virtual	Создание виртуального участника.
Members	GET /api/trips/{tripId}/members	Список участников с memberId.
Expenses	GET /api/trips/{tripId}/expenses	Лента расходов, плательщики и доли по memberId.
Expenses	POST /api/trips/{tripId}/expenses	Добавление расхода и долей.
Balances	GET /api/trips/{tripId}/balances	Оптимизированные долги по memberId.
Balances	POST /api/trips/{tripId}/balances/settle	Ручное подтверждение полной оплаты.
Logs	GET /api/trips/{tripId}/logs	Хронология аудита и подтверждений.
Profile	PUT /api/user/profile	Изменение tbankPaymentUrl , isDarkTheme и email.
Activities	POST /api/trips/{tripId}/activities/import	Импорт сведений из ссылки.

3.4 Примеры дополнительных запросов (УТОЧНЕНИЕ)

POST /api/trips/{tripId}/invite

```
{
  "email": "friend@example.com"
}
```

Альтернативно можно передать phone.

PUT /api/trips/{tripId}/invite/respond

```
{
  "memberId": 11,
  "decision": "Accept"
}
```

POST /api/trips

```
{
  "tripName": "Выходные в Казани",
  "startDate": "2026-11-05",
  "endDate": "2026-11-08",
  "coverUrl": null
}
```

PUT /api/user/profile

```
{
  "tbankPaymentUrl": "https://www.tinkoff.ru/rm/example",
  "isDarkTheme": true
}
```

POST /api/auth/phone/send-code

```
{
  "phone": "+79991234567"
}
```

POST /api/auth/phone/verify-code

```
{
  "phone": "+79991234567",
  "code": "123456"
}
```

Форматы ответов для регистрации, приглашений, списков расходов и ошибок (400/401/403/404/409/422) должны быть окончательно закреплены общей OpenAPI-спецификацией до параллельной реализации. Указанные примеры — предложения актуализированной версии SRS.

3.5 Авиапоиск и ручное сохранение рейса

Поставщик поиска — Aviasales Data API через Travelpayouts; интеграция реализуется Python FastAPI. Выдача представляет кэшированные предложения, не гарантирует доступность тарифа или неизменность цены. Покупка выполняется исключительно на внешнем сайте.

3.5.1 Пользовательский сценарий и границы

1. Авторизованный участник поездки вводит пункт отправления и назначения (IATA-коды) и дату вылета. Система запрашивает предложения и выводит результаты по возрастанию цены в RUB.

2. Пользователь выбирает предложение и переходит по доступной внешней ссылке на Aviasales / сайт продавца. Сам переход не является подтверждением бронирования. В MVP допускается исходная ссылка поиска, сформированная из относительного link и разрешённого домена; партнёрская конвертация через Links API внедряется только после проверки доступов и реального результата.

3. Пользователь может вручную добавить сведения о выбранном рейсе в таймлайн поездки. Такая запись имеет характер плана/сведений пользователя, не подтверждённой системой покупки. Дополнительного автоматического расхода не создаётся.

4. Если поиск не дал результатов, показывается пустое состояние. При сбое провайдера показывается понятная ошибка. Цены и сведения могут изменяться на сайте продавца.

3.5.2 Публичный контракт React ↔ .NET: авиапоиск

GET /api/trips/{tripId}/flights/search?origin=MOW&destination=LED&departureDate=2026-12-15

JWT обязателен; .NET проверяет наличие поездки и участие/доступ пользователя. origin/destination: непустые допустимые IATA-коды, различаются; departureDate: YYYY-MM-DD, не в прошлом. Ответ 200 — предложения по возрастанию цены. Демонстрационный ответ:


```
{
  "tripId": 12,
  "source": "Aviasales",
  "dataType": "cached_offers",
  "currency": "RUB",
  "flights": [
    {
      "origin": "MOW",
      "destination": "LED",
      "departureDate": "2026-12-15",
      "departureAt": "2026-12-15T10:00:00+03:00",
      "airline": "SU",
      "flightNumber": "1234",
      "price": 4500.00,
      "currency": "RUB",
      "transfers": 0,
      "durationMinutes": 95,
      "bookingUrl": "https://www.aviasales.ru/search/..."
    }
  ]
}
```


Публичные ошибки: ProblemDetails с расширением code; 400 — невалидные параметры; 401 — пользовательский JWT; 403 — запрещённое действие; 404 — отсутствующая/скрытая поездка (политика разграничения 403/404 требует финального утверждения); 502 — ошибки Python/Travelpayouts, включая внутренний Python 401; 503 — недоступность Python; 504 — таймаут; неожиданные ошибки C# — 500.

3.5.3 Внутренний контракт .NET ↔ Python: поиск

GET /internal/flights/search?origin=MOW&destination=LED&departureDate=2026-12-15

Python принимает только origin, destination, departureDate; проверяет X-Internal-Api-Key, нормализует кэшированные предложения (RUB), возвращает source/dataType/currency/flights без tripId и offerId. Все ключи предложения присутствуют; неизвестные необязательные сведения — null. C# не передаёт Python пользовательский JWT. Согласованы таймаут C# 30 с и отключение редиректов. Последняя версия Python OpenAPI должна быть приложена к общим контрактам.

3.5.4 Публичный контракт ручного сохранения рейса

POST /api/trips/{tripId}/activities; JWT, проверка доступа к поездке. Рейс сохраняется в существующей Trip_Activity. Для MVP это сведения о запланированном перелёте, а не доказательство оплаты. Пример запроса:

```
{
  "title": "Перелёт Москва — Санкт-Петербург",
  "type": "Flight",
  "description": "Выбранный рейс SU1234; покупка не подтверждена приложением",
  "location": "Москва — Санкт-Петербург",
  "sourceUrl": "https://www.aviasales.ru/search/...",
  "scheduledAt": "2026-12-15T07:00:00Z"
}
```

Ответ 201: {"activityId":45,"tripId":12,"type":"Flight"}. scheduledAt передаётся в UTC, если известно точное время вылета; иначе null. Поле sourceUrl является необязательным при ручном сохранении рейса. Если ссылка предоставлена, она должна быть проверенной HTTPS-ссылкой допустимого домена. Для поддержки отсутствующей ссылки поле TripActivity.SourceUrl изменяется на nullable через отдельную миграцию БД. При импорте активности по URL ссылка остаётся обязательной.

3.5.5 Критерии приёмки базового MVP

Поиск работает по двум IATA-кодам и дате; доступ к поездке проверяется .NET; результаты возвращаются в RUB и сортируются по цене; ссылки открываются во внешнем браузере; пользователь видит предупреждение об изменяемости цены; рейс можно вручную сохранить в таймлайне; пустая выдача и недоступность провайдера обрабатываются; токены и маркеры не раскрываются в клиенте.

3.6 Отслеживание бронирований Travelpayouts — расширенный этап, НЕ входит в обязательный MVP

3.6.1 Проектируемый алгоритм и ответственность

1. .NET создаёт уникальный SubID для конкретной попытки перехода, сохраняет ссылку на пользователя, поездку и снимок выбранного предложения. Один SubID может соответствовать нескольким action_id; связь «один переход = одна покупка» запрещено предполагать.

2. Python формирует партнёрскую ссылку через Links API при наличии подключённой программы, Project ID (trs) и marker; запрашивает статистику через get_fields_list/execute_query и возвращает нормализованные данные.

3. .NET периодически опрашивает Python, учитывает задержки статистики, уникальность событий по action_id в пределах соответствующей программы и хранит историю состояний. Пустой ответ не трактуется как отмена покупки.

4. Состояние paid подтверждает партнёрское действие, но не доказывает, что куплен исходно выбранный рейс и не является электронным билетом. Автоматическое добавление в Trip_Activity допустимо только после проверки типа действия и достаточных деталей маршрута, даты и сегментов; автоматическое создание Expenses по одному статусу запрещено.

3.6.2 Проектные внутренние контракты .NET ↔ Python

POST /internal/affiliate-links — генерация ссылки. Запрос:

```
{
  "subId": "tp_8e7b7a1520c34a87890b84b383a91d77",
  "originalUrl": "https://www.aviasales.ru/search/..."
}
```

Ответ 200:
{"subId":"tp_8e7b7a1520c34a87890b84b383a91d77","partnerUrl":"https://example.com/partner-link"}. Пример URL демонстрационный. Требуются актуальные параметры Links API и проверка соответствия партнёрской программе.

POST /internal/bookings/statistics — получение статистики с диапазоном дат и пагинацией. Пример запроса:

```
{
  "dateFrom": "2026-11-06",
  "dateTo": "2026-11-13",
  "limit": 100,
  "offset": 0
}
```

Демонстрационный нормализованный ответ:

```
{
  "bookings": [
    {
      "subId": "tp_8e7b7a1520c34a87890b84b383a91d77",
      "actionId": "action-987",
      "state": "processing",
      "externalClickId": "click-321",
      "originIata": "MOW",
      "destinationIata": "LED",
      "departDate": "2026-12-15",
      "flightsRaw": null,
```

```
      "priceRub": 4500.00
    }
  ],
  "totalRows": 1
}
```


Поля в примере носят проектный характер. Реальные типы, значения и доступность полей должны быть проверены на записи бронирования. Допускается несколько actionId на один subId. Сопоставление и запись состояния — идемпотентные.

3.7 Самостоятельное указание долей участниками — возможное расширение, НЕ входит в обязательный MVP

В обязательном MVP сохраняется текущая логика добавления расходов: создатель расхода выбирает участников и распределяет сумму поровну либо вручную указывает долю каждого участника («Своя сумма»). Расход сохраняется только при совпадении суммы долей с общей суммой. Действующие API-контракты и ER-модель для этого сценария не изменяются.

В будущей расширенной версии может появиться дополнительный режим «Участники указывают сами»: создатель расхода прикрепляет чек, указывает общую сумму, плательщика и участников, а каждый зарегистрированный участник самостоятельно вводит свою долю. Для виртуальных участников долю указывает уполномоченный пользователь поездки.

4. СОГЛАСОВАНИЕ С ER-МОДЕЛЬЮ И ОГРАНИЧЕНИЯ ДАННЫХ

- Users: user_id; username; phone UNIQUE NOT NULL; email UNIQUE nullable, добавляется в ЛК; tbank_payment_url nullable; is_dark_theme default false. password_hash и парольные маршруты должны быть удалены после согласованной миграции и обновления DTO. Номер нельзя считать надёжно восстановимым без отдельного сценария восстановления доступа.
- Trips: trip_id, trip_name, start_date, end_date, cover_url; валютного поля нет. Active/Future/Past предпочтительно вычислять, не хранить как постоянный статус.
- Trip_Members: member_id; trip_id FK; user_id FK nullable; virtual_name nullable; role; status. Для виртуального участника user_id = null, virtual_name задан.
- Expenses: paid_by_member_id FK → Trip_Members.member_id; trip_id FK → Trips.trip_id; положительная сумма.
- Expense_splits: expense_id FK и member_id FK; UNIQUE(expense_id, member_id); сумма долей равна общей сумме.
- Logs: trip_id FK, user_id FK (для системных событий может быть NULL), action_type, action_message, from_member_id FK nullable, to_member_id FK

nullable, amount nullable, timestamp. Для DebtSettled обязательны оба memberId и положительная сумма; это подтверждённое пользователем событие, а не подтверждение банка.

- Обязательны проверки принадлежности memberId соответствующей поездке при создании расхода/погашении; защита от повторных подтверждений, конкурентных запросов и неподтверждённых участников.